Київський національний університет імені Тараса Шевченка
Факультет комп’ютерних наук та кібернетики
Кафедра інтелектуальних програмних систем

ЗВІТ
до лабораторної роботи №2

Тема: модифікація lexer/parser для Tiger та реалізація AST evaluator

Виконав студент 3-го курсу
Міцкевич Костянтин Олександрович

Київ – 2026

1. Мета роботи

Метою даної роботи є ознайомлення зі структурою навчального компілятора Tiger, розуміння взаємодії лексичного аналізатора, синтаксичного аналізатора та AST, а також практична модифікація skeleton-проекту dragon-tiger.

У межах роботи потрібно було додати підтримку цілочисельних літералів, умовних конструкцій if/then/else, логічного оператора OR, налаштувати пріоритети операторів і реалізувати обчислювач AST для простих виразів. Практичний сенс роботи полягає в тому, щоб побачити, як зміна правил lexer і parser впливає на побудову абстрактного синтаксичного дерева, а також як це дерево можна використати для безпосереднього обчислення результату виразу.

2. Початкові відомості про проект

У лабораторній роботі використовувався skeleton-проект dragon-tiger. Це спрощена реалізація частини компілятора мови Tiger, у якій уже присутні базові класи AST, lexer, parser, driver-програма та механізм виводу побудованого дерева.

Основна ідея роботи з проектом виглядає так: вхідний текст Tiger-програми спочатку обробляється lexer-ом, який розбиває потік символів на токени. Далі parser на основі токенів будує AST. Після цього AST можна або вивести за допомогою ASTDumper, або обійти спеціальним visitor-ом для обчислення значення виразу.

Компонент	**Файл / директорія**	**Призначення**
Lexer	src/parser/tiger_lexer.ll	Розпізнає ключові слова, символи, ідентифікатори, рядки та додані цілочисельні літерали.
Parser	src/parser/tiger_parser.yy	Описує граматику мови Tiger, створює AST-вузли та задає пріоритети операторів.
AST nodes	src/ast/nodes.hh	Містить класи вузлів AST. Цей файл не змінювався, а використовувався як довідковий.
AST dumper	src/ast/ast_dumper.hh / .cc	Виводить побудоване AST у текстовому вигляді для перевірки parser-а.
AST evaluator	src/ast/ast_evaluator.hh	Новий файл, створений для обчислення значення AST.
Driver	src/driver/driver.cc	Головна програма dtiger, яка обробляє параметри запуску та викликає parser/evaluator.

3. Ініціалізація лабораторної роботи

Робота виконувалась в середовищі Ubuntu 24. Спочатку було створено окрему директорію для другої лабораторної роботи та імпортовано skeleton-проєкт dragon-tiger. Далі проєкт було зібрано стандартним для цього skeleton способом.

```
mkdir lab2
cd lab2
wget -qO- www.sifflez.org/lectures/compil/lab2/dragon-tiger.tar.gz | tar zxv
cd dragon-tiger
./configure
make
```

Після успішної збірки було перевірено, що утиліта dtiger запускається та виводить список доступних параметрів. Це потрібно для того, щоб переконатися, що початковий skeleton працює коректно до внесення змін.

```
src/driver/dtiger --help
```

На цьому етапі в списку параметрів ще не було опції -e / --eval. Це означало, що режим обчислення AST ще не реалізований і має бути доданий у процесі виконання лабораторної роботи.

```
user@AX-275180:~/Завантажене/ UNI/compilers/lab2$ cd dragon-tiger
./configure
make
checking for a BSD-compatible install... /usr/bin/install -c
checking whether build environment is sane... yes
checking for a thread-safe mkdir -p... /usr/bin/mkdir -p
checking for gawk... no
checking for mawk... mawk
checking whether make sets $(MAKE)... yes
checking whether make supports nested variables... yes
checking whether make supports nested variables... (cached) yes
checking for bison... yes
checking for bison version >= 2.7... yes
checking for bison... bison -y
checking whether make supports the include directive... yes (GNU style)
checking for gcc... gcc
```

```
user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$ src/driver/dtiger --help
Options:
  -h [ --help ]         describe arguments
  --dump-ast            dump the parsed AST
  --trace-parser        enable parser traces
  --trace-lexer         enable lexer traces
  -v [ --verbose ]      be verbose
  --input-file arg      input Tiger file
```

4. Реалізація лабораторної роботи

4.1. Додавання підтримки цілочисельних літералів

Початковий skeleton не розпізнавав числові літерали як повноцінні вирази. Тому першим кроком було додано підтримку токена INT. Це потрібно для того, щоб parser міг працювати не лише з ідентифікаторами або рядками, а й з арифметичними виразами на основі цілих чисел.

У файлі src/parser/tiger_lexer.ll було додано шаблони intlit і badint. Шаблон intlit відповідає коректним цілим числам, а badint використовується для окремої обробки чисел з провідними нулями, наприклад 012. Такий поділ потрібний, щоб не просто “пропустити” некоректне число, а видати зрозумілу помилку.

```
digit      < ->    [0-9]
nzdigit    <->     [1-9]
intlit     <->     0|{nzdigit}{digit}*
badint     <->     0{digit}+
```

Для перетворення текстового представлення числа у значення використовувалась функція strtol. Додатково перевірялось, чи число не перевищує максимально допустиме значення TIGER_INT_MAX. Це потрібно для того, щоб уникнути некоректної обробки надто великих чисел.

У файлі src/parser/tiger_parser.yy було додано токен INT, створено нетермінал intExpr і підключено його до загального правила expr. У результаті число почало сприйматися parser-ом як повноцінний вираз і перетворюватися на AST-вузол IntegerLiteral.

```
%token <Symbol> ID "id"
%token <Symbol> STRING "string"
%token <int> INT "int"
```

```
user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$ echo "12" | src/driver/dtiger --dump-ast -
12
user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$ echo "0" | src/driver/dtiger --dump-ast -
0
user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$ echo "012" | src/driver/dtiger --dump-ast -
1.1-3: leading zeros are forbidden
user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$
```

```
/* Exprs */

intExpr: INT
  { $$ = new IntegerLiteral(@1, $1); }
;

stringExpr: STRING
  { $$ = new StringLiteral(@1, $1); }
;

var : ID
  { $$ = new Identifier(@1, $1); }
;
```

4.2. Налаштування пріоритетів операторів

Після додавання чисел виникла потреба правильно розбирати арифметичні та логічні вирази. Без налаштування precedence rules parser може будувати неоднозначне або неправильне дерево. Наприклад, вираз 1+2*3 повинен

трактуватися як 1+(2*3), а не як (1+2)*3.

Для цього у файлі tiger_parser.yy було додано правила пріоритетів та асоціативності операторів. Нижчий пріоритет отримали логічні оператори, далі оператори порівняння, потім додавання і віднімання, після них множення та ділення. Найвищий пріоритет отримав унарний мінус.

```
// Declare precedence rules

%nonassoc FUNCTION VAR TYPE DO OF ASSIGN
%nonassoc THEN
%nonassoc ELSE
%left OR
%left AND
%nonassoc EQ NEQ LT GT LE GE
%left PLUS MINUS
%left TIMES DIVIDE
%left UMINUS
```

Ці правила потрібні не для обчислення самих значень, а саме для правильної побудови AST. Якщо дерево побудоване правильно, тоді і ASTDumper, і evaluator працюватимуть з коректною структурою виразу.

```
user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$ echo "1+2*3" | src/driver/dtiger --dump-ast -
(1+(2*3))
user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$ echo "-4+5" | src/driver/dtiger --dump-ast -
((0-4)+5)
user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$ echo "5-2-1" | src/driver/dtiger --dump-ast -
((5-2)-1)
user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$
```

4.3. Додавання конструкцій if / then / else

Наступним етапом було додано підтримку умовних виразів. Це потрібно для того, щоб мова могла описувати розгалуження: залежно від значення умови обирається одна з двох гілок виразу.

У lexer було додано ключові слова if та then. Ключове слово else вже було в skeleton-проєкті. Після цього parser почав отримувати окремі токени IF, THEN і ELSE, а не сприймати ці слова як звичайні ідентифікатори.

У parser було створено правило ifExpr, яке будує AST-вузол IfThenElse. Було реалізовано дві форми: повну if expr then expr else expr та скорочену if expr then expr.

```
;

whileExpr: WHILE expr DO expr { $$ = new WhileLoop(@1, $2, $4); }
;

ifExpr: IF expr THEN expr ELSE expr
  { $$ = new IfThenElse(@1, $2, $4, $6); }
  | IF expr THEN expr %prec THEN
  { $$ = new IfThenElse(@1, $2, $4, new Sequence(nl, std::vector<Expr *>())); }
;

forExpr: FOR ID ASSIGN expr TO expr DO expr
  { $$ = new ForLoop(@1, new VarDecl(@2, $2, boost::none, $4, true), $6, $8); }
;
```

Скорочена форма без else була реалізована через порожню послідовність Sequence. Це дозволило не створювати окремий тип AST-вузла для if без else, а використати вже наявну структуру IfThenElse.

```
user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$ echo "if 1 then 2 else 3" | src/driver/dtiger --dump-ast -
if
  1
 then
  2
 else
  3
user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$ echo "if 1 then 2" | src/driver/dtiger --dump-ast -
if
  1
 then
  2
 else
  (
  )
user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$
```

4.4. Реалізація логічного оператора OR

Оператор OR було реалізовано не як звичайний BinaryOperator, а через IfThenElse. Це зроблено для підтримки короткого обчислення, тобто lazy evaluation. Якщо ліва частина OR істинна, праву частину не потрібно обчислювати, оскільки результат уже відомий.

Вираз a | b перетворюється на логіку: якщо a істинне, результатом є 1, інакше перевіряється b. Якщо b істинне, результатом є 1, інакше 0. Такий підхід дозволяє отримати логічну поведінку OR без додавання нового AST-вузла.

```
  | expr GE expr     { $$ = new BinaryOperator(@2, $1, $3, b_ge); }
  | expr AND expr    {
    $$ = new IfThenElse(@2, $1,
                        new IfThenElse(@3, $3, new IntegerLiteral(nl, 1), new IntegerLiteral(nl, 0)),
                        new IntegerLiteral(nl, 0));
  }
  | expr OR expr     {
    $$ = new IfThenElse(@2, $1,
                        new IntegerLiteral(nl, 1),
                        new IfThenElse(@3, $3, new IntegerLiteral(nl, 1), new IntegerLiteral(nl, 0)));
  }
```

```
● user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$ echo "1|0" | src/driver/dtiger --dump-ast -
 if
   1
  then
   1
  else
   if
     0
    then
     1
    else
     0
○ user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$
```

4.5. Створення AST evaluator

До цього моменту програма могла лише побудувати AST і вивести його через --dump-ast. Для практичної перевірки роботи parser-а було реалізовано AST evaluator - окремий visitor, який обходить дерево та повертає цілочисельний результат.

Файл src/ast/nodes.hh уже містив інтерфейс ConstASTIntVisitor, тому evaluator було реалізовано як клас ASTEvaluator, що наслідує цей інтерфейс. Це дозволило не змінювати наявні AST-вузли, а додати нову поведінку через патерн Visitor.

Evaluator обробляє такі вузли: IntegerLiteral, BinaryOperator, Sequence та IfThenElse. Для IntegerLiteral повертається саме значення числа. Для BinaryOperator спочатку обчислюються ліва і права частини, після чого застосовується відповідна операція. Для IfThenElse обчислюється умова, а потім тільки одна потрібна гілка. Для Sequence результатом є значення останнього обчисленого виразу.

AST-вузол	Що робить evaluator	Для чого це потрібно
IntegerLiteral	Повертає значення цілого числа.	Щоб числа могли бути результатом виразу.
BinaryOperator	Обчислює ліву та праву частину і застосовує операцію.	Щоб працювали +, -, *, / та порівняння.
IfThenElse	Обчислює умову і виконує лише then або else гілку.	Щоб умовні вирази давали правильний результат.
Sequence	Послідовно обчислює вирази та повертає останній результат.	Щоб підтримати вирази у дужках та порожню else-гілку.

4.6. Додавання нового режиму запуску -e / --eval

Після створення evaluator потрібно було надати користувачу спосіб запустити його з командного рядка. Для цього було змінено файл src/driver/driver.cc, який відповідає за обробку CLI-опцій і запуск основної логіки програми.

До списку параметрів було додано опцію -e / --eval. Якщо ця опція передана, після успішного парсингу AST передається в ASTEvaluator, а результат виводиться у стандартний потік виводу.

```
("eval,e", "evaluate the parsed AST")
```

Також було додано перевірку, яка забороняє одночасно використовувати --dump-ast і --eval. Це потрібно тому, що ці режими мають різну мету: перший показує структуру дерева, а другий виводить результат обчислення. Одночасне використання могло б зробити поведінку програми неоднозначною.

```
user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$ src/driver/dtiger --help
Options:
  -h [ --help ]         describe arguments
  --dump-ast            dump the parsed AST
  -e [ --eval ]         evaluate the parsed AST
  --trace-parser        enable parser traces
  --trace-lexer         enable lexer traces
  -v [ --verbose ]      be verbose
  --input-file arg      input Tiger file
```

5. Тестування

Тестування виконувалось у двох режимах роботи утиліти dtiger: режимі побудови AST (--dump-ast) та режимі обчислення виразу (-e). Такий підхід дозволяє окремо перевірити правильність синтаксичного розбору і правильність обчислення вже побудованого AST.

5.1. Перевірка цілочисельних літералів

Ці тести потрібні для перевірки роботи lexer-а і parser-а з числами. Вони показують, що коректні числа стають IntegerLiteral, а некоректний формат із провідним нулем відхиляється.

```
echo "12" | src/driver/dtiger --dump-ast -
echo "0" | src/driver/dtiger --dump-ast -
echo "012" | src/driver/dtiger --dump-ast -
```

Команда / вираз	Що перевіряє	Очікуваний результат
12	Звичайний цілочисельний літерал	AST містить IntegerLiteral зі значенням 12
0	Допустимість числа 0	AST містить IntegerLiteral зі значенням 0
012	Заборона провідних нулів	Повідомлення про помилку leading zeros

```
user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$ echo "12" | src/driver/dtiger --dump-ast -
12
user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$ echo "0" | src/driver/dtiger --dump-ast -
0
user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$ echo "012" | src/driver/dtiger --dump-ast -
1.1-3: leading zeros are forbidden
user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$
```

5.2. Перевірка пріоритетів операторів

Ці тести перевіряють, що parser правильно будує дерево з урахуванням пріоритетів операторів. Це важливо, бо evaluator лише обчислює вже побудовану структуру і не виправляє помилки parser-а.

```
echo "1+2*3" | src/driver/dtiger --dump-ast -
echo "5-2-1" | src/driver/dtiger --dump-ast -
echo "-4+5" | src/driver/dtiger --dump-ast -
```

Вираз	Що перевіряє	Очікуваний AST за змістом
1+2*3	Пріоритет множення над додаванням	1+(2*3)
5-2-1	Лівоасоціативність віднімання	(5-2)-1
-4+5	Коректність унарного мінуса	(0-4)+5

```
user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$ echo "1+2*3" | src/driver/dtiger --dump-ast -
(1+(2*3))
user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$ echo "5-2-1" | src/driver/dtiger --dump-ast -
((5-2)-1)
user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$ echo "-4+5" | src/driver/dtiger --dump-ast -
((0-4)+5)
user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$
```

5.3. Перевірка умовних конструкцій

Ці тести потрібні для перевірки, що parser правильно розпізнає повну і скорочену форму умовного виразу. Особливо важливо перевірити форму без else, оскільки вона була реалізована через порожню Sequence.

```
echo "if 1 then 2 else 3" | src/driver/dtiger --dump-ast -
echo "if 1 then 2" | src/driver/dtiger --dump-ast -
```

Вираз	Що перевіряє	Очікуваний результат
if 1 then 2 else 3	Повна форма умовного виразу	AST-вузол IfThenElse з then та else гілками
if 1 then 2	Скорочена форма без else	IfThenElse з порожньою Sequence в else-гілці

```
user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$ echo "if 1 then 2 else 3" | src/driver/dtiger --dump-ast -
if
  1
 then
  2
 else
  3
user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$ echo "if 1 then 2" | src/driver/dtiger --dump-ast -
if
  1
 then
  2
 else
  (
  )
user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$
```

5.4. Перевірка логічних операторів

Для логічних операторів важливо перевірити не лише числовий результат, а й те, що OR реалізовано через IfThenElse, а не як звичайний BinaryOperator. Це підтверджується виводом AST.

```
echo "1|0" | src/driver/dtiger --dump-ast -
echo "1&0" | src/driver/dtiger --dump-ast -
```

Вираз	Що перевіряє	Очікуваний результат
1|0	Побудову OR через IfThenElse	AST містить умовний вираз
1&0	Поведінку AND	AST містить умовну логіку для короткого обчислення

```
user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$ echo "1|0" | src/driver/dtiger --dump-ast -
if
  1
 then
  1
 else
  if
    0
   then
    1
   else
    0
user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$ echo "1&0" | src/driver/dtiger --dump-ast -
if
  1
 then
  if
    0
   then
    1
   else
    0
 else
  0
user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$
```

5.5. Перевірка режиму обчислення -e / --eval

Після перевірки AST було протестовано режим -e. Ці команди потрібні для підтвердження того, що evaluator правильно обходить дерево і повертає очікувані значення.

```
echo "2+3" | src/driver/dtiger -e -
echo "1+2*3" | src/driver/dtiger -e -
echo "if 1 then 7 else 9" | src/driver/dtiger -e -
echo "if 0 then 7 else 9" | src/driver/dtiger -e -
echo "1|0" | src/driver/dtiger -e -
```

Вираз	Що перевіряє	Очікуваний результат
2+3	Додавання	5
1+2*3	Обчислення з пріоритетами	7
if 1 then 7 else 9	Виконання then-гілки	7
if 0 then 7 else 9	Виконання else-гілки	9
1|0	Обчислення OR	1

```
user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$ echo "2+3" | src/driver/dtiger -e -
5
user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$ echo "1+2*3" | src/driver/dtiger -e -
7
user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$ echo "if 1 then 7 else 9" | src/driver/dtiger -e -
7
user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$ echo "if 0 then 7 else 9" | src/driver/dtiger -e -
9
user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$ echo "1|0" | src/driver/dtiger -e -
1
user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$
```

5.6. Додаткові тестові приклади

Для більш повної перевірки було використано додаткові тести. Вони не є базовими, але дозволяють перевірити вкладеність дужок, оператори порівняння, AND, OR та вкладені умовні вирази.

```
echo "2*(3+4)" | src/driver/dtiger -e -
echo "1<2" | src/driver/dtiger -e -
echo "2=2" | src/driver/dtiger -e -
echo "2<>2" | src/driver/dtiger -e -
echo "1&0" | src/driver/dtiger -e -
echo "0|0" | src/driver/dtiger -e -
echo "if 0 then 1 else if 1 then 2 else 3" | src/driver/dtiger -e -
```

Вираз	Для чого потрібен тест	Очікуваний результат
2*(3+4)	Перевірка дужок та вкладеного виразу	14
1<2	Перевірка оператора порівняння	1
2=2	Перевірка рівності	1
2<>2	Перевірка нерівності	0
1&0	Перевірка AND	0
0|0	Перевірка OR для двох хибних значень	0
if 0 then 1 else if 1 then 2 else 3	Перевірка вкладеного if	2

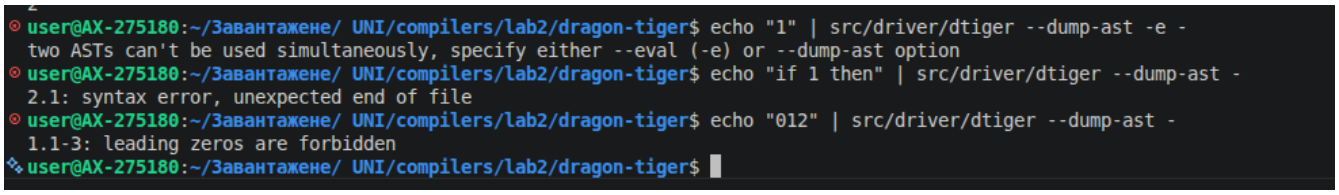
```
user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$ echo "2*(3+4)" | src/driver/dtiger -e -
14
user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$ echo "1<2" | src/driver/dtiger -e -
1
user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$ echo "2=2" | src/driver/dtiger -e -
1
user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$ echo "2<>2" | src/driver/dtiger -e -
0
user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$ echo "1&0" | src/driver/dtiger -e -
0
user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$ echo "0|0" | src/driver/dtiger -e -
0
user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$ echo "if 0 then 1 else if 1 then 2 else 3" | src/drive
2
user@AX-275180:~/Завантажене/ UNI/compilers/lab2/dragon-tiger$
```

5.7. Перевірка помилкових сценаріїв

Окремо було перевірено помилкові сценарії. Це потрібно для того, щоб підтвердити не тільки правильну роботу на коректних виразах, а й адекватну поведінку програми при некоректному введенні.

```
echo "1" | src/driver/dtiger --dump-ast -e -
echo "if 1 then" | src/driver/dtiger --dump-ast -
echo "012" | src/driver/dtiger --dump-ast -
```

Команда / вираз	Що перевіряє	Очікуваний результат
--dump-ast -e	Заборона одночасного запуску двох режимів	Повідомлення про помилку
if 1 then	Неповний умовний вираз	Синтаксична помилка
012	Некоректний формат числа	Помилка leading zeros



6. Результати роботи

У результаті виконання лабораторної роботи було змінено чотири основні частини проєкту. У lexer було додано розпізнавання чисел та нових ключових слів. У parser було додано нові grammar-правила, precedence rules та побудову нових AST-структур. У src/ast було створено evaluator для обчислення значення AST. У driver було додано новий режим запуску -e / --eval.

Файл	Що змінено	Навіщо це потрібно
src/parser/tiger_lexer.ll	Додано правила для INT, badint, if, then.	Щоб lexer міг передавати parser-у нові коректні токени.
src/parser/tiger_parser.yy	Додано INT, intExpr, ifExpr, OR через IfThenElse, precedence rules.	Щоб parser міг будувати правильне AST для нових виразів.
src/ast/ast_evaluator.hh	Створено ASTEvaluator на базі ConstASTIntVisitor.	Щоб мати можливість обчислювати AST, а не тільки виводити його.
src/driver/driver.cc	Додано опцію -e / --eval і виклик evaluator.	Щоб запускати обчислення з командного рядка.

7. Висновок

У ході виконання лабораторної роботи було успішно модифіковано skeleton-проєкт dragon-tiger. Було додано підтримку цілочисельних літералів, умовних конструкцій if/then/else та if/then, реалізовано логічний оператор OR через

IfThenElse, налаштовано пріоритети операторів і створено AST evaluator для базових виразів.

Після внесення змін утиліта dtiger почала працювати у двох основних режимах: режимі побудови AST та режимі безпосереднього обчислення значення виразу. Це показує повний шлях обробки вхідного тексту: від розпізнавання токенів lexer-ом до побудови AST parser-ом і подальшого обходу дерева evaluator-ом.

Результати тестування підтвердили коректність реалізованих змін: арифметичні вирази враховують пріоритети операторів, умовні конструкції правильно будуються в AST, логічні оператори працюють через умовну логіку, а режим -e повертає очікувані результати. Таким чином, мету лабораторної роботи було досягнуто.